Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Model Poisoning Attacks to Federated Learning based on Fake Clients

Mani Ghahremani

mani.ghahremani@port.ac.uk

University of Portsmouth

Alan Metwally

University of Portsmouth

Rahim Taheri

University of Portsmouth

Research Article

Keywords: Federated Learning, Model Poisoning, Adversarial Attacks, Network Security

Posted Date: September 2nd, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7109247/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Model Poisoning Attacks to Federated Learning based on Fake Clients

Mani Ghahremani[a,1], **Alan Metwally**[b,1], **Rahim Taheri**[c,1]

[1]School of Computing, University of Portsmouth, Buckingham Building, Lion Terrace, Portsmouth PO1 3HE, UK

Received: date / Accepted: date

Abstract The increasing use of decentralized and anonymous networks creates vast amounts of darknet traffic, offering opportunities to enhance network security by detecting threats, filtering malicious activity, and identifying anomalies through improved traffic classification. Federated Learning (FL) presents a promising approach for decentralized data processing, allowing models to be trained across distributed devices while preserving data privacy. However, FL is vulnerable to poisoning attacks, where adversarial clients can degrade the performance of the global model. In this paper, we utilize a rich dataset that captures encrypted darknet traffic to develop new methods for defending against model poisoning attacks. We propose novel attack strategies based on fake clients and gradient inversion: Model Poisoning Attack based on Fake Clients (MPAF), Gradient Descent Inversion Attack (GDIA), and Selective Aggregation Poisoning Attack (SAPA). Alongside these attacks, we introduce two defense strategies: Adaptive Weighting in Aggregation (AWA) and Statistical Outlier Filtering (SOF). Experimental results show that attacks like GDIA can drastically reduce accuracy to 0% and the MPAF attack reduces accuracy to approximately 32.38%. The AWA defense notably restores accuracy under GDIA to around 80.95% and under MPAF to about 93.33%, clearly outperforming SOF. After refining the attack implementations by strengthening the base model for MPAF and reducing the intensity of GDIA, MPAF became significantly stronger, bringing accuracy down to 0%. However, GDIA exhibited more controlled degradation, with AWA defense still effectively stabilizing accuracy at approximately 72.06%.

[a]e-mail: mani.ghahremani@port.ac.uk
[b]e-mail: up2203391@myport.ac.uk
[c]e-mail: rahim.taheri@port.ac.uk

Keywords Federated Learning · Model Poisoning · Adversarial Attacks · Network Security

1 Introduction

Traditional machine learning (ML) approaches often require centralized data collection, raising significant privacy concerns among users [1]. To address these concerns, Federated Learning (FL) has emerged as a promising paradigm that allows the training of ML models across decentralized devices while keeping the data locally on each device [2, 3].

FL offers a decentralized approach to model training, where local models are trained on individual devices and only the model updates are shared with a central server, which aggregates these updates to form a robust global model. Various aggregation methods have been developed to enhance performance and robustness, such as FedAvg [4], which averages the weights of locally trained models to facilitate efficient model convergence.

Despite its advantages, FL is not without challenges. The distributed nature of FL introduces new vulnerabilities, particularly in terms of model robustness and security. One of the most critical threats to FL is the potential for adversarial attacks, particularly model poisoning attacks [5, 6].

Model poisoning attacks in FL occur when adversaries manipulate the model updates sent from local devices to the central server with the intention of degrading the performance of the global model. These attacks can be highly effective due to the lack of visibility into the training data and the reliance on aggregate updates, which can conceal malicious modifications [7, 8]. Various strategies, including backdoor attacks, have been

explored to exploit these vulnerabilities, leading to significant concerns about the reliability and safety of FL systems [9–11].

Considering these challenges, improving the robustness of FL against such adversarial attacks has become a focal point of recent research. Several approaches have been proposed to mitigate these risks, including the development of robust aggregation functions [5, 6, 12], secure multi-party computation [13], and the use of blockchain technologies to ensure the integrity of model updates [14, 15].

In this work, we utilize the CIC-Darknet2020 dataset [16], which is particularly relevant for classifying darknet traffic, including both VPN and Tor data. The dataset amalgamates ISCXTor2016 with ISCXVPN2016 and includes various traffic categories such as Audio-Stream, Browse, P2P, and VOIP, which are significant in detecting unsought network interactions.

Our goal is then to explore new strategies for enhancing the resilience of FL models against adversarial attacks. We will do this by first designing and implementing novel model poisoning attacks that work by injecting fake clients. Namely, these are Model Poisoning Attack based on Fake Clients (MPAF), Gradient Descent Inversion Attack (GDIA), and Selective Aggregation Poisoning Attack (SAPA). Next, we propose two novel defence strategies, namely, Adaptive Weighting in Aggregation (AWA) and Statistical Outlier Filtering for Update Validation (SOF), to mitigate the impact of the aforementioned attacks. By systematically assessing the effectiveness of both attack and defence mechanisms, our study aims to contribute actionable insights to the design of secure and robust FL systems.

The remainder of this paper is organized as follows: Section 2 discusses related work on adversarial attacks and defences in federated learning. Section 3 formally defines the problem alongside our implementation. Our proposed attack and defence strategies are described respectively in sections 4 and 5. In Section 6, we present the experimental setup and results, demonstrating the effectiveness of our methods. Later in Section 7, we discuss the implications of our findings and their relevance to the broader context of federated learning security. Finally, Section 8 concludes the paper and outlines future research directions.

2 Related Work

Common attacks in FL include backdoor attacks, gradient attacks, and model poisoning attacks. Backdoor attacks inject malicious triggers into model updates to manipulate predictions when specific inputs are encountered [9–11]. Gradient attacks exploit the gradients shared during training to infer private data [17, 18]. Model poisoning attacks involve adversaries sending malicious updates to degrade the global model's performance [7, 8]. One specific form of model poisoning is the label flipping attack, where adversaries intentionally alter the labels in their local datasets to mislead the global model [19, 20].

We focus on aggregation methods and defences against these attacks. We classify these methods into three categories: static, dynamic, and hybrid aggregation strategies, and discuss the defences associated with each.

Static aggregation methods rely on the straightforward averaging of model weights across clients. Introduced by Google in 2016, FedAvg remains one of the most widely used methods in FL due to its simplicity and effectiveness in scenarios where client data is IID (Independent and Identically Distributed) [4]. However, it faces challenges when data is non-IID across clients, as this can lead to model divergence.

To defend against model poisoning attacks in static aggregation, robust aggregation functions have been proposed, including Byzantine-tolerant gradient descent algorithms [12] and sparsification methods that reduce the impact of malicious updates [6].

Dynamic aggregation methods adapt the aggregation process based on client-specific characteristics, adjusting for variations in data distribution and client behaviour. Examples include FedProx, which introduces a proximal term to stabilise training in heterogeneous settings [21], and FedMA, which employs matched averaging to align and average the hidden elements of neural networks for better performance in diverse data environments [22].

In dynamic settings, differential privacy mechanisms have been applied to ensure the confidentiality of updates. The Distributed Discrete Gaussian Mechanism is an example of how noise is added to local updates to protect client data [23].

Hybrid methods combine features of both static and dynamic aggregation, offering flexibility in adapting to client data distributions while maintaining the efficiency of static methods. These methods often employ a mix of robust aggregation functions and secure aggregation protocols. For instance, FoolsGold adjusts the aggregation weights to mitigate Sybil-based attacks by downweighting clients whose updates are overly similar, thus preventing malicious collusion [24].

Other secure aggregation techniques include homomorphic encryption and blockchain, which ensure the integrity and privacy of model updates by making it computationally infeasible for adversaries to access or

modify sensitive data during the aggregation process [13–15].

Adaptive aggregation methods, such as FedProx and FedMA, have also been proposed to adjust the aggregation process based on client-specific model updates and data distributions, enhancing robustness and generalization in diverse environments [21, 22].

3 Background and Problem Definition

In this section, we first present the fundamental concepts of Federated Learning. This is followed by an overview of our system implementation and the threat model and assumptions considered in this study.

3.1 Federated Learning Overview

FL is a distributed machine learning paradigm where multiple clients collaborate to train a shared model while keeping their data local. The training process involves a central server and multiple participating clients, each with their local dataset. The global model, typically a neural network with parameters denoted as θ, is distributed to all clients. For a more comprehensive overview of FL and its challenges, we refer the reader to [25].

The training process occurs in rounds, where the server distributes the current global model parameters θ to selected clients. Each client then trains the model on their local data using an optimization algorithm, such as stochastic gradient descent (SGD) and computes the gradient of the loss function with respect to the model parameters. In our implementation, we used the Adam optimizer [26], an adaptive learning rate optimization algorithm. Clients then compute gradients of their loss function $\nabla L(\theta_c)$. The server aggregates the updates using FedAvg as shown in Equation (1).

$$\theta_{new} = \theta_{old} - (l_r/N) * \sum(\nabla L(\theta_c)) \tag{1}$$

where l_r is the learning rate and N is the number of participating clients.

3.2 System Implementation

Our implementation leverages the Flower framework, a robust platform designed specifically for federated learning research and applications. We employ a multi-layer neural network architecture trained on the CIC-Darknet2020 dataset ([16]), which combines ISCXTor2016 and ISCXVPN2016 to provide comprehensive coverage of darknet traffic patterns. The dataset encompasses various traffic categories essential for network security analysis, including Audio-Stream, Browse, P2P, and VOIP communications.

Key static features from the CIC-Darknet2020 dataset are carefully selected to ensure consistency across federated clients and enhance the model's robustness against adversarial manipulation. The federated model's parameters, such as the number of clients, communication rounds, learning rate, and batch size, are tuned to balance performance and resilience in the face of attacks.

3.3 Threat Model and Assumptions

Our threat model considers various attack scenarios in FL, focusing on the vulnerabilities that arise from the distributed nature of the training process.

More specifically, we assume that our adversary has the following capabilities:

- Control a subset of participating clients
- Access and modify local model parameters
- Launch targeted attacks during the training process

We also make the following assumptions about our system:

- The central server is trusted and cannot be compromised
- Communication channels between server and clients are secure
- Adversaries have limited computational resources
- Non-malicious clients honestly follow the protocol
- Attacks are launched during the training phase only
- Model evaluation metrics include loss, accuracy and the size of the validation data

4 Proposed Attack Strategies Against Federated Learning

In this section, we present several attack strategies designed to exploit vulnerabilities in FL systems.These strategies aim to degrade the performance of the global model or introduce targeted biases, emphasizing the importance of robust defenses. Specifically, we detail three approaches: Model Poisoning Attack based on Fake Clients (MPAF) discussed in Section 4.1 and Algorithm 1, Gradient Descent Inversion Attack (GDIA) in Section 4.2 and Algorithm 2, and Selective Aggregation Poisoning Attack (SAPA) in Section 4.3 and Algorithm 3. Each method leverages different aspects of the FL framework to achieve its goals, as described in subsequent subsections. The time complexity of the attacks is discussed in Section 4.1.4.

4.1 Base Method: Model Poisoning Attack based on Fake Clients (MPAF)

The Model Poisoning Attack based on Fake Clients (MPAF) is designed to degrade the accuracy of the global model by injecting fake clients that provide manipulated updates. This approach does not require any genuine local training data, distinguishing it from other attacks that rely on compromising genuine clients.

MPAF involves initializing a base model with random parameters and creating fake clients to send detrimental updates to the federated learning system. The attack also involves generating plausible fake metrics to evade detection by the server. The steps are as follows. For a formal definition, we have provided Algorithm 1.

4.1.1 Initialization

- The attacker initializes a base model w' with random parameters.
- The attacker simulates several fake clients. These clients participate in the training rounds by receiving the global model and sending malicious updates. No genuine data is required for this process.

4.1.2 Training Rounds

1. Each fake client receives the current global model, w_t, from the server.
2. The fake client calculates the update direction by computing the difference between the base model and the current global model, $d = w' - w_t$.
3. The update is then scaled by a factor λ to enhance its impact on the global model.
4. Fake clients generate plausible fake metrics (e.g., low loss and high accuracy) to make their updates appear legitimate.

4.1.3 Aggregation at Server

- The server aggregates updates from all clients (both genuine and fake). The updates from the fake clients, due to their scaling, disproportionately affect the global model.
- This process repeats for multiple rounds, with each round designed to drag the global model closer to the characteristics of the base model.

4.1.4 Time Complexity of Attack Strategies

Since all attack stratgies described in this paper are based on the same principle of model poisoning, we can state that the time complexity of the attacks is $O(T \cdot m \cdot |w|)$, where T is the number of rounds, m is the number of fake clients, and $|w|$ is the model size. Note that this complexity applies for MPAF (Algorithm 1), GDIA (Algorithm 2), and SAPA (Algorithm 3).

Algorithm 1 Model Poisoning Attack based on Fake Clients (MPAF)

Input: Base model parameters w', current global model w_t, scaling factor λ, number of fake clients m, number of rounds T
Output: Poisoned global model w_T
for $t = 1$ **to** T **do**
  Server broadcasts w_t to all fake clients
  for $i = 1$ **to** m **do**
    Compute direction: $d_i \leftarrow w' - w_t$
    Scale update: $g_{t,i} \leftarrow \lambda \cdot d_i$
    Generate fake metrics: $\ell_i \sim \mathcal{U}(0.1, 0.5)$ and $a_i \sim \mathcal{U}(0.8, 0.95)$
    Send $(g_{t,i}, \ell_i, a_i)$ to the server
  end for
  Aggregate: $w_{t+1} \leftarrow w_t + \sum_{i=1}^{m} g_{t,i}$
end for
return w_T

Note that the direction of client i, d_i, is computed as the difference between the base model and the current global model, $d_i = w' - w_t$. Additionally, in Algorithm 1, the fake metrics are generated using the uniform distribution function $\mathcal{U}(a, b)$, which generates random values between a and b. The fake loss is generated from a uniform distribution between 0.1 and 0.5, which represents a relatively low loss value. This is done to make the fake client updates appear plausible and not too far from the expected performance of a legitimate client. Similarly, the fake accuracy is generated from a uniform distribution between 0.8 and 0.95, indicating a high accuracy value.

4.1.5 Key Innovations

- MPAF focuses on creating a poor-performing base model and driving the global model toward it. This ensures that the attack's impact is maximized without the need for genuine local data.
- Unlike other attacks, MPAF operates without access to genuine client data, reducing the dependency on external compromises or datasets.
- By generating plausible fake metrics such as low loss and high accuracy, the attack remains under the radar of traditional detection mechanisms.
- As with other attacks, MPAF can scale effectively with multiple fake clients, enhancing its ability to influence the global model during aggregation.

4.2 Gradient Descent Inversion Attack (GDIA)

The Gradient Descent Inversion Attack (GDIA) strategically manipulates the model updates to degrade the model's performance subtly and effectively. GDIA leverages the concept of inverting gradient updates to push the global model toward a state of poor performance. Unlike direct malicious updates, GDIA's approach is to calculate and then invert gradients based on a crafted loss function that, if optimized, would degrade the model's performance. The formal steps are presented in Algorithm 2.

4.2.1 Steps in GDIA

1. Design a loss function that minimizes the error rate for the model.
2. Compute the gradient of this loss function with respect to the current global model parameters.
3. Invert the direction of these gradients to degrade model performance.
4. Scale the inverted gradients and add noise to blend them with legitimate updates, evading detection based on statistical properties.
5. Submit these manipulated gradients to the federated learning server during the standard model update phase.

Algorithm 2 Gradient Descent Inversion Attack (GDIA)

Input: Malicious loss function L', current global model w_t, number of fake clients m, number of rounds T
Output: Degraded global model w_T
for $t = 1$ **to** T **do**
  Server broadcasts w_t to all fake clients
  for $i = 1$ **to** m **do**
    Compute gradient: $g_{t,i} \leftarrow \nabla L'(w_t)$
    Invert gradient: $g_{t,i} \leftarrow -g_{t,i}$
    Optional: Adjust update as $g_{t,i} \leftarrow \alpha\, g_{t,i} + \eta$, where α is a scaling factor and η is noise
    Send $g_{t,i}$ to the server
  end for
  Aggregate: $w_{t+1} \leftarrow w_t + \sum_{i=1}^{m} g_{t,i}$
end for
return w_T

4.2.2 Key Innovations

- The use of inverted gradients makes this attack hard to detect using conventional outlier detection methods.
- The attack can be customized to target specific vulnerabilities in the model, offering flexibility in how the performance degradation is applied.

4.3 Selective Aggregation Poisoning Attack (SAPA)

The Selective Aggregation Poisoning Attack (SAPA) introduces a novel approach to model poisoning by selectively manipulating parameters during the aggregation process. This method focuses on degrading model performance by targeting parameters that deviate significantly from the statistical norm. See Algorithm 3 for the detailed implementation steps.

SAPA exploits the aggregation phase of FL by modifying parameters outside a defined range (mean $\pm\ k \cdot$ std). Parameters above this range are scaled down, while those below are scaled up, ensuring subtle yet effective degradation of the global model.

4.3.1 Steps in SAPA

1. For each parameter, compute the mean and standard deviation to identify upper and lower bounds based on a scaling factor k.
2. Adjust parameters outside the range (mean $\pm\ k \cdot$ std). Scale down values exceeding the upper bound, and scale up those below the lower bound.
3. Camouflage the malicious updates by matching their statistical profile with legitimate updates and generating plausible metrics.
4. Submit the crafted updates to the federated learning server along with fake metrics (e.g., low loss and high accuracy).

Similar to the MPAF attack (see Algorithm 1), the SAPA attack described in Algorithm 3, calculates the fake metrics (low loss and high accuracy, ℓ_i and a_i respectively) using the uniform distribution with the same ranges as in MPAF.

4.3.2 Key Innovations

- Parameters outside the statistical norm (mean $\pm\ k \cdot$ std) are selectively modified, ensuring focused and subtle attacks.
- By camouflaging updates and generating plausible metrics, SAPA minimizes detection risks.
- The attack exploits the fact that networks have an increased dependency on those outliers, amplifying the impact of the poisoned updates.

5 Defense Strategies Against Model Poisoning Attacks in Federated Learning

In this section, we outline defence mechanisms designed to mitigate the impact of model poisoning attacks similar to the ones described in Section 4. These strategies

Algorithm 3 Selective Aggregation Poisoning Attack (SAPA)

Input: Current global model w_t, statistical factor k, attack scale γ, number of fake clients m, number of rounds T
Output: Poisoned global model w_T
for $t = 1$ **to** T **do**
  Server broadcasts w_t to all fake clients
  for $i = 1$ **to** m **do**
    for each parameter $p \in w_t$ **do**
      Compute $\mu_p \leftarrow \text{mean}(p)$ and $\sigma_p \leftarrow \text{std}(p)$
      Set upper bound: $p_{\text{upper}} \leftarrow \mu_p + k\sigma_p$
      Set lower bound: $p_{\text{lower}} \leftarrow \mu_p - k\sigma_p$
      if $p > p_{\text{upper}}$ **then**
        $p' \leftarrow \frac{p}{\gamma}$
      else if $p < p_{\text{lower}}$ **then**
        $p' \leftarrow p \cdot \gamma$
      else
        $p' \leftarrow p$
      end if
      Compute update: $\Delta p \leftarrow p' - p$
    end for
    Form update $g_{t,i}$ from $\{\Delta p\}$ over all parameters
    Generate fake metrics: $\ell_i \sim \mathcal{U}(0.1, 0.5)$ and $a_i \sim \mathcal{U}(0.8, 0.95)$
    Send $(g_{t,i}, \ell_i, a_i)$ to the server
  end for
  Aggregate: $w_{t+1} \leftarrow w_t + \sum_{i=1}^{m} g_{t,i}$
end for
return w_T

aim to enhance the resilience of the global model by detecting and mitigating adversarial updates. Specifically, we present the approaches Adaptive Weighting in Aggregation (AWA) in Section 5.1, and Statistical Outlier Filtering for Update Validation (SOF) in Section 5.2. Each method leverages different aspects of the federated learning framework to strengthen its defenses, as described in subsequent subsections.

5.1 Adaptive Weighting in Aggregation (AWA)

Adaptive Weighting is designed to mitigate the impact of malicious client updates in FL. By assigning dynamic performance-based weights to client contributions during aggregation, this approach aims to enhance the robustness of FL systems. We have provided below an overview of the Adaptive Weighting in Aggregation strategy, along with the formal definition in Algorithm 4.

Let the global model at round t be W_t, and the update submitted by client i be $\Delta W_{i,t}$. The goal is to compute a weighted aggregation of client updates to form the global model for round $t+1$. Adaptive Weighting modifies the aggregation as follows:

$$W_{t+1} = W_t + \frac{1}{\sum_{i=1}^{K} \tau_i} \sum_{i=1}^{K} \tau_i \Delta W_{i,t}, \tag{2}$$

where K is the total number of participating clients, τ_i is the trust score assigned to client i, dynamically computed to reflect the quality and reliability of its updates. Also, $\Delta W_{i,t}$ is the update vector submitted by client i at round t.

The trust score τ_i is calculated by the server based on the client's contribution performance. Specifically, the server evaluates the performance of each client's model update by testing it on a held-out validation dataset. The mean and standard deviation of the resulting accuracies across all clients are then used to compute the trust score:

$$\tau_i = \begin{cases} 1, & \text{if } \frac{A_i - \mu_A}{\sigma_A} \geq 0, \\ 0, & \text{otherwise,} \end{cases} \tag{3}$$

where A_i is the accuracy of client i's update, μ_A is the mean accuracy across all clients, and σ_A is the standard deviation of accuracies.

The adaptive aggregation mechanism operates in three key stages: The server dynamically computes trust scores τ_i for all clients during each training round based on update performance. It then aggregates client updates using Equation (2), prioritizing contributions from high-performing clients. The aggregated global model is validated using a held-out test dataset to ensure robustness and resilience against adversarial updates.

The Adaptive Weighting in Aggregation strategy is formally defined in Algorithm 4 and it can be verified that the time complexity of this algorithm is $O(T \cdot K \cdot |w|)$, where T is the number of rounds, K is the number of clients, and $|w|$ is the model size. This complexity arises from the need to compute the trust scores and aggregate the updates in each round. The trust score calculation involves computing the mean and standard deviation of the accuracies, which is $O(K)$. The aggregation of the updates involves summing K vectors of length $|w|$, leading to a complexity of $O(K \cdot |w|)$. Thus, the overall complexity for each round is $O(K \cdot |w|)$, and across T rounds, it becomes $O(T \cdot K \cdot |w|)$.

5.1.1 Key Innovations

- AWA introduces a trust scoring system based on the evaluation of client updates against a validation dataset, dynamically adjusting client contributions during aggregation.
- Clients with low-performing updates are effectively excluded from influencing the global model.
- Prioritizing high-performing clients ensures robustness against malicious updates.

Algorithm 4 Adaptive Weighting in Aggregation (AWA)

Input: Initial global model parameters W_0, client updates $\{\Delta W_{i,t}\}_{i=1}^{K}$, corresponding accuracies $\{A_i\}_{i=1}^{K}$, number of clients K, number of rounds T
Output: Global model parameters W_T
for $t = 1$ **to** T **do**
  Broadcast: Server sends W_t to all K clients.
  for $i = 1$ **to** K **do**
    Client i computes local update $\Delta W_{i,t}$ and reports performance A_i.
  end for
  Compute:

$$\mu_A \leftarrow \text{mean}(\{A_i\}_{i=1}^{K}), \quad \sigma_A \leftarrow \text{std}(\{A_i\}_{i=1}^{K})$$

  for $i = 1$ **to** K **do**
    Compute trust score τ_i according to Equation (3)
  end for
  Aggregate W_{t+1} according to Equation (2)
end for
return W_T

5.2 Statistical Outlier Filtering for Update Validation (SOF)

The Statistical Outlier Filtering (SOF) defence applies an outlier filtering mechanism to client updates, rejecting those that deviate significantly from expected parameter distributions. This approach ensures the robustness of the global model by mitigating the influence of malicious or anomalous client updates. Below is a detailed overview of the Statistical Outlier Filtering strategy, along with the formal definition in Algorithm 5.

- For each model parameter, percentile-based bounds (e.g., 5th to 95th percentiles) are computed using updates from all clients in a round. These bounds reflect typical parameter value ranges observed across updates.
- Each client's update is analysed to determine whether any parameter values lie outside the established bounds. Updates with excessive outliers are flagged as anomalous and excluded from aggregation.
- To prevent over-filtering, SOF ensures that no more than 10% of client updates are removed in any given round, prioritizing the retention of updates closest to the established bounds.

The Statistical Outlier Filtering strategy is formally defined in Algorithm 5. The time complexity of this algorithm is $O(L \cdot K \cdot |w_l| \cdot \log(K \cdot |w_l|))$, where L is the number of layers, K is the number of clients, and $|w_l|$ is the size of each layer. The logarithmic factor arises from the need to compute percentiles for each layer across all clients. The filtering procedure runs in $O(K \cdot |w|)$ per round, leading to a total complexity of $O(T \cdot K \cdot |w|)$ across T rounds, where $|w|$ is the number of parameters across all layers.

Algorithm 5 Statistical Outlier Filtering for Update Validation (SOF)

Input: Client updates $\{\Delta W_{i,t}\}_{i=1}^{K}$, lower percentile p_{low}, upper percentile p_{high}, maximum removable proportion δ, number of clients K
Output: Filtered client update set $\mathcal{S} \subseteq \{1, \ldots, K\}$
for each model layer l **do**
  Collect all parameter values $\{w_{i,t}^{(l)}\}_{i=1}^{K}$ from $\Delta W_{i,t}$
  Compute bounds:

$$L^{(l)} \leftarrow \text{percentile}(\{w_{i,t}^{(l)}\}, p_{\text{low}}), \quad U^{(l)} \leftarrow \text{percentile}(\{w_{i,t}^{(l)}\}, p_{\text{high}})$$

end for
Initialize $\mathcal{S} \leftarrow \{1, 2, \ldots, K\}$
for $i = 1$ **to** K **do**
  for each layer l **do**
    if any element of $\Delta W_{i,t}^{(l)}$ satisfies $\Delta W_{i,t}^{(l)} < L^{(l)}$ or $\Delta W_{i,t}^{(l)} > U^{(l)}$ **then**
      Remove i from $\mathcal{S}$ and **break** inner loop.
    end if
  end for
end for
if $K - |\mathcal{S}| > \delta K$ **then**
  Compute distance d_i for each removed client i, where d_i is the sum over layers of deviations beyond $L^{(l)}$ or $U^{(l)}$.
  Retain the $K - \delta K$ clients with the smallest d_i, and update $\mathcal{S}$ accordingly.
end if
return $\{\Delta W_{i,t}\}_{i \in \mathcal{S}}$

5.2.1 Key Innovations

- SOF employs percentile-based bounds to identify and exclude anomalous updates effectively.
- Ensures that no more than a set proportion of updates are removed, preventing over-filtering.
- Dynamically adjusts to varying parameter distributions across training rounds, ensuring relevance and reliability.

6 Experimental Evaluation

In this section, after providing an overview of the metrics used to evaluate our models, we present the results of our evaluation. Initially, we demonstrate the performance of our FL model under the 10 different scenarios and highlight the strengths and shortcomings of the applied attacks and defenses. After observing specific limitations in the original implementations, we subsequently introduce modifications to address these

shortcomings. We conclude this section with an analysis of the refined attack implementations, comparing their relative effectiveness and stability improvements.

6.1 Test metrics

To evaluate our federated learning model on the test set, we used a selection of common performance metrics. The confusion matrix summarizes the model's predictions, categorizing outcomes into true positives (correctly identified intrusions), false positives (incorrectly identified normal activities as intrusions), true negatives (correctly identified normal activities), and false negatives (missed intrusions).

Based on these outcomes, we calculated accuracy, which indicates the proportion of correct predictions across all cases:

$$Accuracy = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}.$$

Additionally, we measured precision, which reflects how many of the positively classified instances were actually positive:

$$Precision = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

and recall, which assesses the model's capability to correctly identify actual positive cases:

$$Recall = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

To balance precision and recall, we also computed the F1-score, defined as the harmonic mean of these two metrics:

$$F1\text{-}Score = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

For our experiments, we used the hyperparameters listed in Table 1.

Table 1: Hyperparameters

Hyperparameter	Value	Description
`NUM_CLIENTS`	50	Total clients
`NUM_FAKE_CLIENTS`	15	Fake clients
`NUM_ROUNDS`	100	Training rounds
`BATCH_SIZE`	32	Batch size
`EPOCHS`	5	Training epochs
`TRAIN_PARTITION`	0.7	Training data fraction
`VALID_PARTITION`	0.33	Validation data fraction
`TEST_PARTITION`	0.66	Test data fraction

In total, we ran 10 experiments, each with 100 rounds of training. The experiments were designed to evaluate the performance of our FL model under various attack scenarios and the effectiveness of our proposed defence mechanisms. The experiments were conducted using the Flower framework, which provides a robust platform for federated learning research and applications.

The first experiment involved training the FL model without any adversarial influence (labeled `None-None`) to establish a baseline. Subsequent experiments introduced each of the three attacks (MPAF, GDIA, SAPA) independently and then combined each attack with one of our two defences (AWA, SOF). The results of these experiments are shown in Figures 1 and 2, which illustrate the accuracy and loss of the model over 100 server rounds, respectively. In both figures, the lines are labeled as {*Attack*}-{*Defence*}. For instance, `None-None` indicates no attack and no defence, while `MPAF-AWA` indicates the MPAF attack with the AWA defence.

In Figure 2, most lines appear flat due to GDIA's devastating effects. The standout curves are `GDIA-None`, which climbs toward 7 by the final rounds, and `GDIA-SOF`, which slowly rises to around 1. All other scenarios remain nearly flat and close to 0 in comparison.

Table 2: Metrics of the final round (100) of each experiment. Note that "Acc" is accuracy and its values, alongside Precision, Recall, and F1, are expressed as percentages. Loss is expressed in the same units as the training loss.

Strategy	Acc	Loss	Precision	Recall	F1
None-None	94.60	0.13	79.95	76.92	76.26
SAPA-None	91.11	0.20	80.54	77.11	77.14
GDIA-None	0.00	6.70	0.00	0.00	0.00
MPAF-None	32.38	3.69	8.87	10.71	7.92
SAPA-AWA	92.70	0.19	77.72	74.46	74.06
GDIA-AWA	80.95	1.04	50.19	40.30	39.89
MPAF-AWA	93.33	0.18	77.10	75.29	74.33
SAPA-SOF	90.48	0.27	80.70	77.24	75.82
GDIA-SOF	0.63	1.10	0.03	5.00	0.00
MPAF-SOF	55.87	1.67	29.67	27.81	22.66

Table 2 confirms that GDIA was excessively powerful, with accuracy dropping to 0% and loss reaching approximately 6.7. On average, accuracy without defenses was 41.6%, whereas AWA recovered it to 88.9% and SOF to only 48.99%, highlighting SOF's limited effectiveness against GDIA.

6.2 Modifications to the Attacks

To provide a more realistic evaluation of adversarial scenarios, we refined the implementations of MPAF and GDIA attacks. Previously, the global model utilized

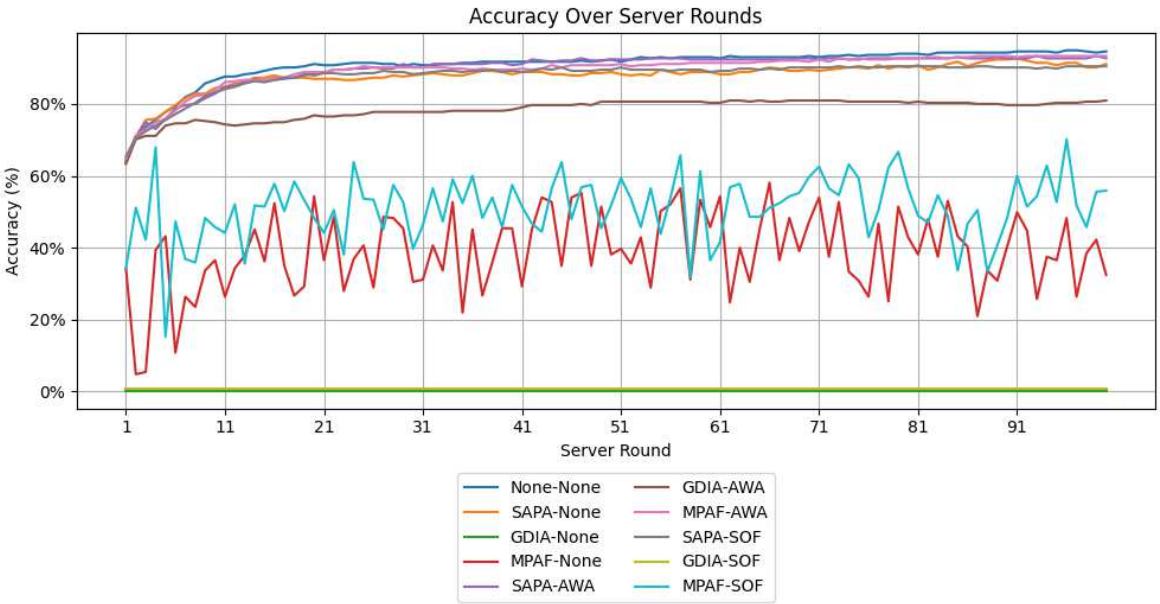

Fig. 1: Accuracy (%) over 100 server rounds.

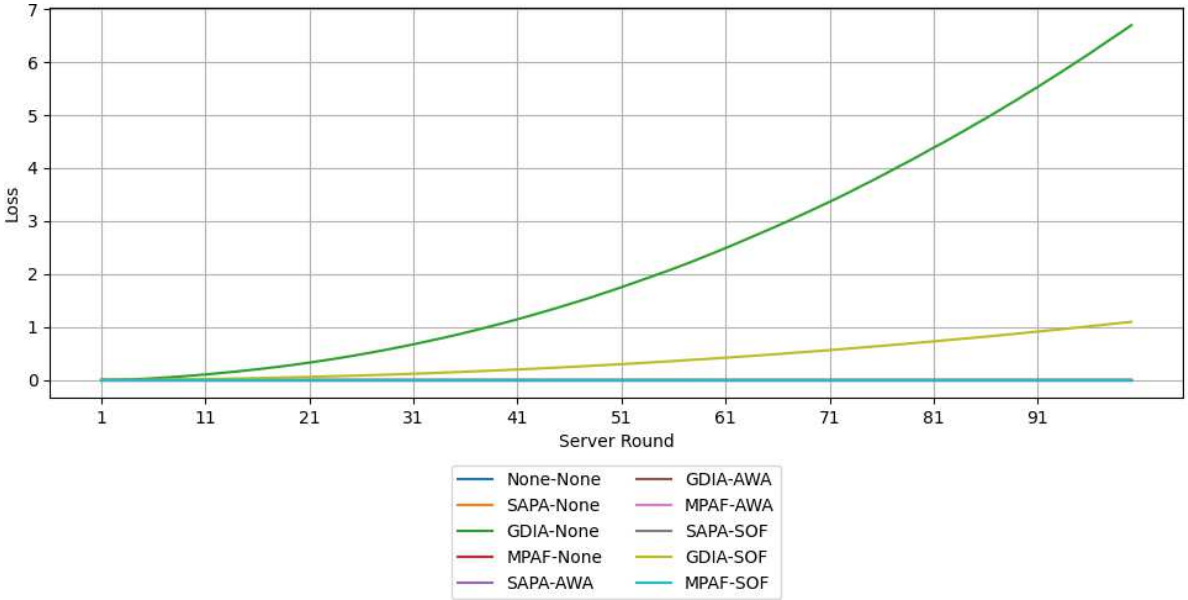

Fig. 2: Loss over 100 server rounds.

random parameters as the initial base model. However, we upgraded to a more potent base model derived from the degraded state after 100 rounds under the GDIA attack. Additionally, we reduced the intensity of GDIA by scaling its loss manipulation factor from -1 to -0.5. This adjustment aims to induce a more gradual and realistic degradation in model performance rather than an abrupt collapse to 0% accuracy.

Figures 3 and 4 illustrate the impact of these modifications. After removing the inherent randomness from MPAF, its behavior became notably more stable, resulting in clearer and more predictable performance trends. Furthermore, employing the updated base model significantly enhanced the attack strength of MPAF, leading to a greater reduction in accuracy compared to the original implementation. Conversely, the weakened GDIA attack now exhibits a more moderate effect. Specifically, the loss under the GDIA-None scenario has significantly decreased. These adjustments offer a more nuanced and balanced evaluation of both

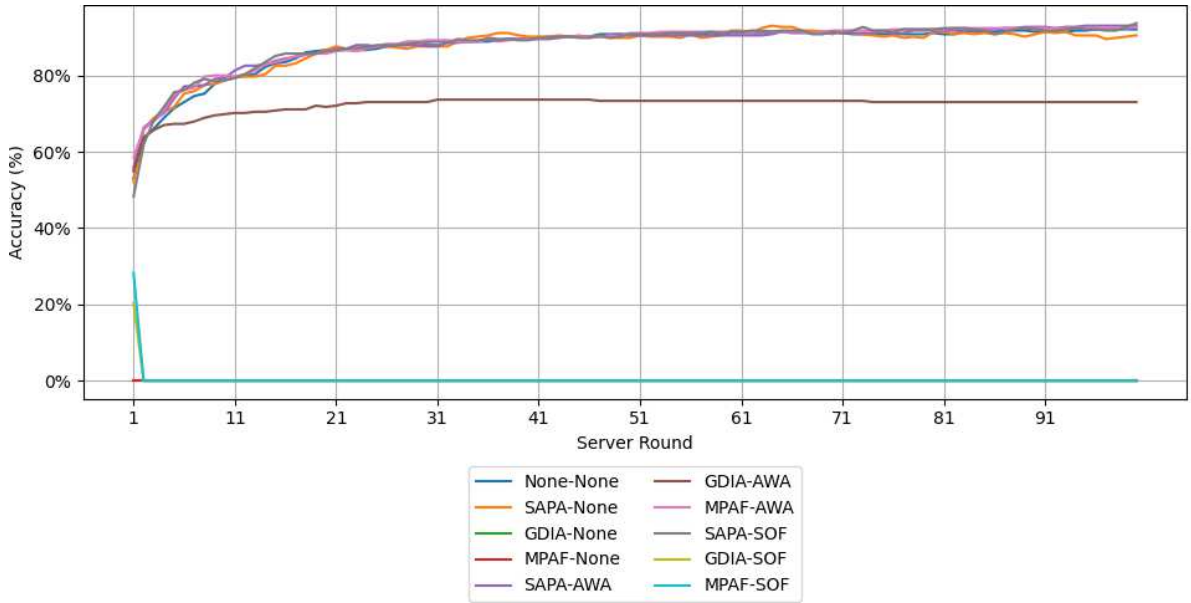

Fig. 3: Accuracy (%) after the updates to MPAF and GDIA.

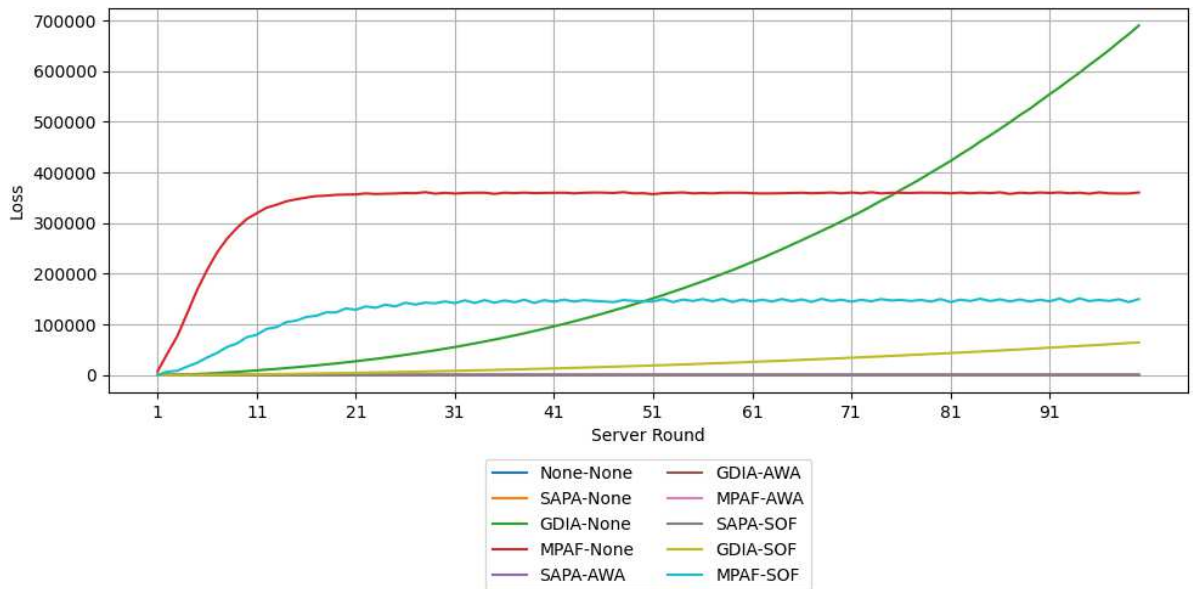

Fig. 4: Loss after the updates to MPAF and GDIA.

the attacks' effectiveness and the corresponding defense mechanisms.

The results after modifying the implementations of MPAF and GDIA are summarized in Table 3, highlighting the refined attacks' effects on the model's performance metrics.

7 Discussion

Section 6 demonstrated that the poisoning attacks MPAF and GDIA degrade federated learning performance. AWA consistently outperformed SOF in recovering model accuracy across all attack strategies (see Figures 1, 2 and Table 2).

As shown in Section 6, the MPAF attack, which moves the global model toward a predefined base model, caused pronounced instability in model performance when no defence was applied, with accuracy fluctuating unpredictably between roughly 20% and 60% (Figure 1, Table 2). This volatility stems from the fact that the initial base model was randomly generated.

Table 3: Metrics of the final round (100) of each experiment after modifications. Note that "Acc" is accuracy and its values, alongside Precision, Recall, and F1, are expressed as percentages. Loss is expressed in the same units as the training loss.

Strategy	Acc	Loss	Prec.	Rec.	F1
None-None	94.29	0.18	90.29	91.99	88.91
SAPA-None	89.52	0.24	84.92	84.31	83.21
GDIA-None	0.00	599383.32	0.00	0.00	0.00
MPAF-None	0.00	359234.93	0.00	0.00	0.00
SAPA-AWA	91.43	0.21	75.74	76.32	75.12
GDIA-AWA	72.06	1.49	15.50	21.61	17.88
MPAF-AWA	93.02	0.20	82.37	86.31	81.93
SAPA-SOF	92.38	0.19	84.19	86.15	82.50
GDIA-SOF	0.95	43468.54	0.04	4.35	0.08
MPAF-SOF	0.00	146969.60	0.00	0.00	0.00

After refining the attack implementation (Section 6.2), replacing the random base with a trained one eliminated this randomness and resulted in a consistent collapse of accuracy to 0% under MPAF-None (Table 3). When the AWA defence was applied (MPAF-AWA), the model's performance rapidly recovered to near-baseline levels by down-weighting poisoned updates and amplifying legitimate contributions (Figure 3, Table 3). In contrast, SOF's normalization approach provided only marginal benefit against MPAF, as merely scaling poisoned parameters did little to counteract their directional effect.

The GDIA attack, inverting the loss gradient, proved the most devastating, reducing accuracy to 0% under GDIA-None in the original experiments (Figure 1, Table 2). After weakening GDIA's inversion factor (Section 6.2), the attack's impact became less abrupt, though accuracy still fell to 0% and loss increased substantially (Figure 4). AWA successfully mitigated GDIA by testing each update's accuracy before aggregation, restoring model accuracy to approximately 72% (Table 3). SOF, however, remained ineffective, as inverted gradients lie outside expected parameter distributions and thus cannot be filtered without discarding most legitimate updates.

SAPA (which shifts outlier weights toward the center) had the mildest impact when applied early, reducing accuracy by less than 10% compared to the no-attack baseline (Section 6). This limited effect occurs because early application functions similarly to dropout, diminishing the model's reliance on extreme weights.

Interestingly, SAPA-None occasionally improved accuracy over the baseline (Table 2), likely by removing harmful outliers and acting as a regularizer. Both AWA and SOF effectively restored accuracy to near-baseline levels against SAPA by either down-weighting anomalous updates or filtering them out.

7.1 Discussion of Attack Modifications

Section 6.2 refined MPAF and GDIA to reflect more realistic adversarial capabilities. By using a degraded base model for MPAF and scaling GDIA's loss inversion factor to -0.5, the revised attacks present controlled yet potent impacts on model performance. Figures 3-4 and Table 3 show that modified MPAF collapses accuracy to 0% without defence, while the weakened GDIA yields a gradual degradation that AWA mitigates to 72% accuracy. These changes enable a balanced assessment of attack strength and defence resilience, underscoring the need for future defences to anticipate increasingly sophisticated poisoning strategies.

8 Conclusions and Future Work

In this paper, we introduced three model poisoning attack methods for Federated Learning: Model Poisoning Attack based on Fake Clients (MPAF), Gradient Descent Inversion Attack (GDIA), and Selective Aggregation Poisoning Attack (SAPA). We also presented two novel defensive strategies: Adaptive Weighting in Aggregation (AWA) and Statistical Outlier Filtering (SOF). Experimental evaluation demonstrated the varying effectiveness of these attacks and showed AWA to be particularly effective at restoring model performance after attacks, while SOF provided limited improvements.

The attack models considered in this paper were evaluated in a setting where the global model was trained without any dropout. Introducing dropout into training as a possible avenue of future work could reduce the effectiveness of some of these attacks by diminishing the network's reliance on specific weights with large magnitudes. Specifically, the SAPA attack would likely be significantly weakened due to its dependence on targeting outlier weights. The GDIA attack could also see a slight reduction in effectiveness, as dropout would introduce noise into gradient computations, making inversion slightly more difficult. However, the MPAF attack would likely remain unaffected since it relies on repeatedly injecting malicious updates whenever specific weights become active again. Future research could empirically evaluate these hypotheses and investigate further defensive mechanisms that could enhance robustness against the different attack types discussed in this paper.

References

1. J.P. Albrecht, European Data Protection Law Review **2**, 287 (2016). URL https://api.semanticscholar.org/CorpusID:86604540
2. Q. Yang, Y. Liu, T. Chen, Y. Tong, ACM Trans. Intell. Syst. Technol. **10**(2) (2019). DOI 10.1145/3298981. URL https://doi.org/10.1145/3298981
3. J. Liu, J. Huang, Y. Zhou, X. Li, S. Ji, H. Xiong, D. Dou, Knowledge and Information Systems **64**, 1 (2022). DOI 10.1007/s10115-022-01664-x
4. B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A.y. Arcas, in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, *Proceedings of Machine Learning Research*, vol. 54, ed. by A. Singh, J. Zhu (PMLR, 2017), *Proceedings of Machine Learning Research*, vol. 54, pp. 1273–1282. URL https://proceedings.mlr.press/v54/mcmahan17a.html
5. A.N. Bhagoji, S. Chakraborty, P. Mittal, S. Calo, in *Proceedings of the 36th International Conference on Machine Learning*, *Proceedings of Machine Learning Research*, vol. 97, ed. by K. Chaudhuri, R. Salakhutdinov (PMLR, 2019), *Proceedings of Machine Learning Research*, vol. 97, pp. 634–643. URL https://proceedings.mlr.press/v97/bhagoji19a.html
6. A. Panda, S. Mahloujifar, A.N. Bhagoji, S. Chakraborty, P. Mittal, in *Proceedings of The 25th International Conference on Artificial Intelligence and Statistics*, *Proceedings of Machine Learning Research*, vol. 151, ed. by G. Camps-Valls, F.J.R. Ruiz, I. Valera (PMLR, 2022), *Proceedings of Machine Learning Research*, vol. 151, pp. 7587–7624. URL https://proceedings.mlr.press/v151/panda22a.html
7. X. Cao, N.Z. Gong, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops* (2022), pp. 3396–3404
8. S. Nabavirazavi, R. Taheri, M. Ghahremani, S.S. Iyengar, *Model Poisoning Attack Against Federated Learning with Adaptive Aggregation* (Springer Nature Switzerland, Cham, 2024), pp. 1–27. DOI 10.1007/978-3-031-49803-9_1. URL https://doi.org/10.1007/978-3-031-49803-9_1
9. H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.y. Sohn, K. Lee, D. Papailiopoulos, in *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2020), NIPS '20
10. C. Xie, K. Huang, P.y. Chen, B. Li, in *International Conference on Learning Representations* (2020). URL https://api.semanticscholar.org/CorpusID:213447399
11. E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, V. Shmatikov, in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, *Proceedings of Machine Learning Research*, vol. 108, ed. by S. Chiappa, R. Calandra (PMLR, 2020), *Proceedings of Machine Learning Research*, vol. 108, pp. 2938–2948. URL https://proceedings.mlr.press/v108/bagdasaryan20a.html
12. P. Blanchard, E.M. El Mhamdi, R. Guerraoui, J. Stainer, in *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2017), NIPS'17, pp. 118–128
13. A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, R.M. Parizi, IEEE Transactions on Information Forensics and Security **19**, 6693 (2024). DOI 10.1109/TIFS.2024.3420126
14. Y. Miao, Z. Liu, H. Li, K.K.R. Choo, R.H. Deng, IEEE Transactions on Information Forensics and Security **17**, 2848 (2022). DOI 10.1109/TIFS.2022.3196274
15. R. Aluvalu, S. Kumaran V N, M. Thirumalaisamy, S. Basheer, E. Ali Aldhahri, S. Selvarajan, PeerJ Comput Sci **9**, e1308 (2023). DOI 10.7717/peerj-cs.1308
16. A.H. Lashkari, G. Kaur, A. Rahali, in *Proceedings of the 2020 10th International Conference on Communication and Network Security* (Association for Computing Machinery, New York, NY, USA, 2021), pp. 1–13. DOI 10.1145/3442520.3442521. URL https://doi.org/10.1145/3442520.3442521
17. L. Zhu, Z. Liu, S. Han, in *Advances in Neural Information Processing Systems*, vol. 32, ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, R. Garnett (2019), vol. 32. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/60a6c4002cc7b29142def8871531281a-Paper.pdf
18. J. Geiping, H. Bauermeister, H. Dröge, M. Moeller, in *Advances in Neural Information Processing Systems*, vol. 33, ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Curran Associates, Inc., 2020), vol. 33, pp. 16,937–16,947. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/c4ede56bbd98819ae6112b20ac6bf145-Paper.pdf
19. M. Fang, X. Cao, J. Jia, N. Gong, in *29th USENIX Security Symposium (USENIX Security 20)* (USENIX Association, 2020), pp. 1605–

1622. URL https://www.usenix.org/conference/usenixsecurity20/presentation/fang

20. S.A. Khowaja, P. Khuwaja, K. Dev, A. Antonopoulos. Spin: Simulated poisoning and inversion network for federated learning-based 6g vehicular networks (2022). URL https://arxiv.org/abs/2211.11321
21. H. Li, K. Acharya, P. Richtárik. The power of extrapolation in federated learning (2024). URL https://arxiv.org/abs/2405.13766
22. H. Wang, M. Yurochkin, Y. Sun, D.S. Papailiopoulos, Y. Khazaeni, CoRR **abs/2002.06440** (2020). URL https://arxiv.org/abs/2002.06440
23. P. Kairouz, Z. Liu, T. Steinke, arXiv preprint arXiv:2102.06387 (2022). URL https://arxiv.org/abs/2102.06387
24. C. Fung, C.J.M. Yoon, I. Beschastnikh, in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)* (USENIX Association, San Sebastian, 2020), pp. 301–316. URL https://www.usenix.org/conference/raid2020/presentation/fung
25. J. Wen, Z. Zhang, Y. Lan, Z. Cui, J. Cai, W. Zhang, International Journal of Machine Learning and Cybernetics **14**(2), 513 (2023). DOI 10.1007/s13042-022-01647-y. URL https://doi.org/10.1007/s13042-022-01647-y
26. D.P. Kingma, J. Ba, arXiv preprint arXiv:1412.6980 (2017). URL https://arxiv.org/abs/1412.6980